Ad-hoc 乱讲

注意到其实我并没有注意到什么

Rosaya

USTC

Outline

1. Overview 2
2. ICPC 2025 EC Online Contest (I) 6
   - 2.1 Creating Chaos 7
   - 2.2 Robot 9
   - 2.3 Walk 11
3. ICPC 2025 EC Online Contest (II) 14
   - 3.1 Square Permutation I 15
   - 3.2 Square Permutation II 18
   - 3.3 Reconstruct the tree 20
4. ICPC 2025 Chengdu Regional Contest . 22
   - 4.1 Following Arrows 23
   - 4.2 Meeting for Meals 26
5. ICPC 2025 Nanjing Regional Contest . . 29
   - 5.1 Wow, It's Yesterday Six Times More 30
   - 5.2 What, More Kangaroos? 34
   - 5.3 Bucket Bonanza 36
   - 5.4 Trajan Algorithm 38
6. CCPC 2025 Zhengzhou Site 40
   - 6.1 Diameter of a Tree 41
   - 6.2 Subrectangle Count 43
   - 6.3 Parentheses and Swapping 45
7. ICPC 2025 EC Final Contest 48
   - 7.1 Strategic Stones 49
   - 7.2 Redundancy Refrain 51
   - 7.3 Logical Resonance 53

Outline

1. Overview 2
2. ICPC 2025 EC Online Contest (I) 6
   - 2.1 Creating Chaos 7
   - 2.2 Robot 9
   - 2.3 Walk 11
3. ICPC 2025 EC Online Contest (II) 14
   - 3.1 Square Permutation I 15
   - 3.2 Square Permutation II 18
   - 3.3 Reconstruct the tree 20
4. ICPC 2025 Chengdu Regional Contest . 22
   - 4.1 Following Arrows 23
   - 4.2 Meeting for Meals 26
5. ICPC 2025 Nanjing Regional Contest . . 29
   - 5.1 Wow, It's Yesterday Six Times More 30
   - 5.2 What, More Kangaroos? 34
   - 5.3 Bucket Bonanza 36
   - 5.4 Trajan Algorithm 38
6. CCPC 2025 Zhengzhou Site 40
   - 6.1 Diameter of a Tree 41
   - 6.2 Subrectangle Count 43
   - 6.3 Parentheses and Swapping 45
7. ICPC 2025 EC Final Contest 48
   - 7.1 Strategic Stones 49
   - 7.2 Redundancy Refrain 51
   - 7.3 Logical Resonance 53

这些题目是从我这个赛季做过的所有比较有趣的题目里挑的，里面有银牌题、金牌题，也有可能有杯题。

当然不会每道题质量都很高（），但是你也不能说场上做到出的不好的题就不做了对吧。

如果你能想明白这份题单里的每道题那就足够在如今的 OI 和 ACM 环境下取得比较好的成绩了。

这些题目是从我这个赛季做过的所有比较有趣的题目里挑的，里面有银牌题、金牌题，也有可能有杯题。

当然不会每道题质量都很高（），但是你也不能说场上做到出的不好的题就不做了对吧。

如果你能想明白这份题单里的每道题那就足够在如今的 OI 和 ACM 环境下取得比较好的成绩了。

题目按**时间顺序排列**，相邻的题目直接**并没有直接关系**，请不要以此来猜测解法。

这些题目是从我这个赛季做过的所有比较有趣的题目里挑的，里面有银牌题、金牌题，也有可能有杯题。

当然不会每道题质量都很高（），但是你也不能说场上做到出的不好的题就不做了对吧。

如果你能想明白这份题单里的每道题那就足够在如今的 OI 和 ACM 环境下取得比较好的成绩了。

题目按**时间顺序排列**，相邻的题目直接**并没有直接关系**，请不要以此来猜测解法。

为了防止大家读不懂题面，这里的所有题意均为中文，并去除了一些没什么用的语句。

- https://qoj.ac/contest/2513/problem/14302
- https://qoj.ac/contest/2513/problem/14306
- https://qoj.ac/contest/2513/problem/14308
- https://qoj.ac/contest/2524/problem/14319
- https://qoj.ac/contest/2524/problem/14320
- https://qoj.ac/contest/2524/problem/14323
- https://qoj.ac/contest/2567/problem/14711
- https://qoj.ac/contest/2567/problem/14718
- https://qoj.ac/contest/2581/problem/14801

- https://qoj.ac/contest/2581/problem/14802
- https://qoj.ac/contest/2581/problem/14807
- https://qoj.ac/contest/2581/problem/14810
- https://qoj.ac/contest/2661/problem/15304
- https://qoj.ac/contest/2661/problem/15310
- https://qoj.ac/contest/2661/problem/15311
- https://qoj.ac/contest/3295/problem/16331
- https://qoj.ac/contest/3295/problem/16336
- https://qoj.ac/contest/3295/problem/16339

Outline

1. Overview 2
2. ICPC 2025 EC Online Contest (I) 6
   2.1 Creating Chaos 7
   2.2 Robot 9
   2.3 Walk 11
3. ICPC 2025 EC Online Contest (II) 14
   3.1 Square Permutation I 15
   3.2 Square Permutation II 18
   3.3 Reconstruct the tree 20
4. ICPC 2025 Chengdu Regional Contest . 22
   4.1 Following Arrows 23
   4.2 Meeting for Meals 26
5. ICPC 2025 Nanjing Regional Contest . . 29
   5.1 Wow, It's Yesterday Six Times More 30
   5.2 What, More Kangaroos? 34
   5.3 Bucket Bonanza 36
   5.4 Trajan Algorithm 38
6. CCPC 2025 Zhengzhou Site 40
   6.1 Diameter of a Tree 41
   6.2 Subrectangle Count 43
   6.3 Parentheses and Swapping 45
7. ICPC 2025 EC Final Contest 48
   7.1 Strategic Stones 49
   7.2 Redundancy Refrain 51
   7.3 Logical Resonance 53

2.1 Creating Chaos

选出 $[n]$ 的一个大小为 k 的子集，记剩余的元素为 $a_1, a_2, \ldots, a_{n-k}$，最小化：

$$\sum_{i=1}^{n-k} \sum_{j=i+1}^{n-k} \gcd(|a_i - a_j|, n)$$

构造你选出的大小为 k 的子集。

$1 \le k \le n \le 1000$。

2.1 Creating Chaos

发现 $[k]$ 即为答案。下面我们证明 $a = [n - k]$ 时是所求式最小的解。

2.1 Creating Chaos

发现 $[k]$ 即为答案。下面我们证明 $a = [n-k]$ 时是所求式最小的解。

考虑对每个 d 统计 $|a_i - a_j| \equiv 0 (\bmod d)$ 的数量，发现其实就是将 a_i 按 $\bmod d$ 划分为了 d 个等价类，每个等价类的贡献是下凸的二次函数，显然 $[n-k]$ 能使 $|a_i - a_j| \equiv 0 (\bmod d)$ 的数量最小。

2.1 Creating Chaos

发现 $[k]$ 即为答案。下面我们证明 $a = [n-k]$ 时是所求式最小的解。

考虑对每个 d 统计 $|a_i - a_j| \equiv 0 (\bmod d)$ 的数量，发现其实就是将 a_i 按 $\bmod d$ 划分为了 d 个等价类，每个等价类的贡献是下凸的二次函数，显然 $[n-k]$ 能使 $|a_i - a_j| \equiv 0 (\bmod d)$ 的数量最小。

同时又有若 $x|y$，那么 $\gcd(x,n) \le \gcd(y,n)$，也就是说按 d 从大到小归纳，可以证明对于任意 $[d,n]$ 的贡献均最小，那么也就是原式答案最小。

复杂度 $O(n)$，但 checker 需要 $O(n^2)$ 完成。

2.2 Robot

交互题，整张棋盘初始为白色，故障机器人出现在 $s = (x, y)$，每轮你可以将一个格子染为黑色，随后机器人移动到第一象限且与当前位置八连通的一个白色格子上，如果机器人无法移动则你赢。

请你在 $T = 1000$ 轮内获胜，$1 \le s_x, s_y \le 20$。

2.2 Robot

有点像那个六边形警察抓小偷的游戏。

2.2 Robot

有点像那个六边形警察抓小偷的游戏。

先规划下 1000 次染色怎么用。

先把机器人框在 $N \times N$ 的矩形里，使用 $2N$ 步，后续每次将机器人所在的空间对半切开，共花费 $N + \frac{N}{2} + \frac{N}{2} + \frac{N}{4} + \frac{N}{4} + ... = 3N$ 步，那么我们取 $N = 200$。

2.2 Robot

有点像那个六边形警察抓小偷的游戏。

先规划下 1000 次染色怎么用。

先把机器人框在 $N \times N$ 的矩形里，使用 $2N$ 步，后续每次将机器人所在的空间对半切开，共花费 $N + \frac{N}{2} + \frac{N}{2} + \frac{N}{4} + \frac{N}{4} + ... = 3N$ 步，那么我们取 $N = 200$。

先围一个“大概”的边界出来，可以选择每隔两个格子染色一个，这样这个缝不会很大，我们可以快速修补。

角落比较危险，先修补一下。统计一下大约此时用了 $\frac{N}{3} + \frac{N}{3} + 39 = 171$ 步。

2.2 Robot

有点像那个六边形警察抓小偷的游戏。

先规划下 1000 次染色怎么用。

先把机器人框在 $N \times N$ 的矩形里，使用 $2N$ 步，后续每次将机器人所在的空间对半切开，共花费 $N + \frac{N}{2} + \frac{N}{2} + \frac{N}{4} + \frac{N}{4} + \ldots = 3N$ 步，那么我们取 $N = 200$。

先围一个“大概”的边界出来，可以选择每隔两个格子染色一个，这样这个缝不会很大，我们可以快速修补。

角落比较危险，先修补一下。统计一下大约此时用了 $\frac{N}{3} + \frac{N}{3} + 39 = 171$ 步。

后面机器人靠近哪个边界堵哪个边界即可，限制非常松。

围出边界之后内部再切割即可。

2.3 Walk

给定 K 个矩形，第 i 个矩形的左下角和右上角的坐标分别为 $(x_{i,1}-0.5, y_{i,1}-0.5), (x_{i,2}+0.5, y_{i,2}+0.5)$，你要规划一条从 $(0,0)$ 到 (N,M) 的路径，只能**向右和向上**走（也就是格线游走）。

若你的路径同时经过第 i 个矩形的**左右边界**，那么需要花费 a_i 的代价；若你的路径同时经过第 i 个矩形的**上下边界**，那么需要花费 b_i 的代价。最小化你的路径的代价。

$1 \le N, M \le 50, 1 \le K \le 2000, 0 \le x_{i,1} \le x_{i,2} \le N, 0 \le y_{i,1} \le y_{i,2} \le M$。

2.3 Walk

先考虑想个办法刻画格线游走。

2.3 Walk

先考虑想个办法刻画格线游走。

这里我们记第 i 条竖线是 $x = i - 0.5$，第 j 条横线是 $y = j - 0.5$，那么一个合法的格线游走一定穿过 N 条竖线，M 条横线各一次。

记穿过第 i 条竖线的坐标为 $(i - 0.5, x_i)$，穿过第 j 条横线的坐标为 $\left(y_j, j - 0.5\right)$，那么一定要求 $0 \le x_1 \le x_2 \le \ldots \le x_N \le M, 0 \le y_1 \le y_2 \le \ldots \le y_M \le N$，且对于第 i 条竖线第 j 横线，有 $x_i \ge j$ 和 $y_j \ge i$ **恰好满足一个**。

2.3 Walk

先考虑想个办法刻画格线游走。

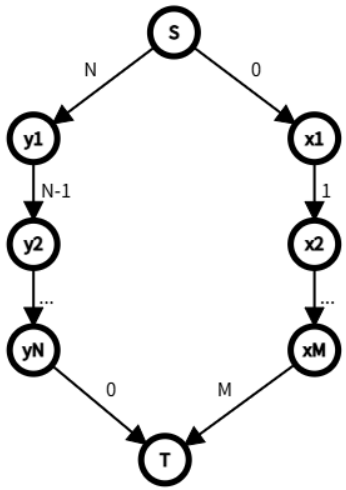
这里我们记第 i 条竖线是 $x = i - 0.5$，第 j 条横线是 $y = j - 0.5$，那么一个合法的格线游走一定穿过 N 条竖线，M 条横线各一次。

记穿过第 i 条竖线的坐标为 $(i - 0.5, x_i)$，穿过第 j 条横线的坐标为 $(y_j, j - 0.5)$，那么一定要求 $0 \le x_1 \le x_2 \le \ldots \le x_N \le M, 0 \le y_1 \le y_2 \le \ldots \le y_M \le N$，且对于第 i 条竖线第 j 横线，有 $x_i \ge j$ 和 $y_j \ge i$ **恰好满足一个**。

不难发现如果没有行列交错的限制，那么就是一个很简单的切糕模型，但是现在有行列限制的话我们怎么处理呢？

一个小巧思就是我们选择倒过来建立横线的切糕，也就是说，我们 $S \to T$ 中间的 $N + 1$ 条边的顺序不再是 $0, 1, \ldots, N$，而是 $N, N - 1, \ldots, 0$。

2.3 Walk



这样建图后，后续的限制和矩形的额外贡献都变得简单了起来，然后跑 Dinic 即可，复杂度 $O(n^3m^3 + n^2m^2k)$ 反正能过。

Outline

1. Overview 2
2. ICPC 2025 EC Online Contest (I) 6
   2.1 Creating Chaos 7
   2.2 Robot 9
   2.3 Walk 11
3. ICPC 2025 EC Online Contest (II) 14
   3.1 Square Permutation I 15
   3.2 Square Permutation II 18
   3.3 Reconstruct the tree 20
4. ICPC 2025 Chengdu Regional Contest . 22
   4.1 Following Arrows 23
   4.2 Meeting for Meals 26
5. ICPC 2025 Nanjing Regional Contest . . 29
   5.1 Wow, It's Yesterday Six Times More 30
   5.2 What, More Kangaroos? 34
   5.3 Bucket Bonanza 36
   5.4 Trajan Algorithm 38
6. CCPC 2025 Zhengzhou Site 40
   6.1 Diameter of a Tree 41
   6.2 Subrectangle Count 43
   6.3 Parentheses and Swapping 45
7. ICPC 2025 EC Final Contest 48
   7.1 Strategic Stones 49
   7.2 Redundancy Refrain 51
   7.3 Logical Resonance 53

3.1 Square Permutation I

给定两个排列 p, q，排列长度均为奇数 n，对每个 i 决策以下三项中的**恰好一项执行**：

1. 什么都不做，花费 0 的代价。
2. 执行 $p_i \leftarrow p_i^2$ **或** $q_i \leftarrow q_i^2$，花费 x_i 的代价。
3. 执行 $p_i \leftarrow p_i^2$ **和** $q_i \leftarrow q_i^2$，花费 y_i 的代价。

求最小代价使得修改后的序列 p, q 的中位数分别为 A, B，无解输出 -1。

多测，$1 \le n \le 10^5, 1 \le x_i \le y_i \le 10^4$。

3.1 Square Permutation I

把 $< A$ 的记为 -1，$> A$ 的记为 1，那么 A 是中位数当且仅当：

- A 出现在序列中。
- 1 和 -1 的出现次数相同。

平方只会让数增大，也就是说只有从 $< A$ 变为 $\geq A$ 是有效的。

B 同理。

3.1 Square Permutation I

把 $< A$ 的记为 -1，$> A$ 的记为 1，那么 A 是中位数当且仅当：

- A 出现在序列中。
- 1 和 -1 的出现次数相同。

平方只会让数增大，也就是说只有从 $< A$ 变为 $\geq A$ 是有效的。

B 同理。

先分析下第一条，由于 $1 \leq x_i \leq y_i$ 存在，能不操作就不操作一定更优。也就是如果 $A \leq n$ 那么 A 一定不变，否则 $\sqrt{A}$ 必须为整数并且必须平方。

我们称 -1 变 1 为一次操作，那么我们可以处理出对 p, q 需要各多少次操作，记为 k, l。

3.1 Square Permutation I

不同决策的能分三类：

1. 花费 x_i 操作 p。
2. 花费 x_i 操作 q。
3. 花费 x_i 操作 p 或 q，或花费 y_i 同时操作 p 和 q。

这样的话，如果我们枚举花费 y_i 同时操作 p 和 q 的数量 m，那么就是一个简单的费用流模型。

3.1 Square Permutation I

不同决策的能分三类：

1. 花费 x_i 操作 p。
2. 花费 x_i 操作 q。
3. 花费 x_i 操作 p 或 q，或花费 y_i 同时操作 p 和 q。

这样的话，如果我们枚举花费 y_i 同时操作 p 和 q 的数量 m，那么就是一个简单的费用流模型。

简单画一下这个费用率，发现操作比较简单可以模拟费用率，先令 m 最大判断是否有界，虽然逐渐减小 m 并统计答案即可。

复杂度 $O(n \log n)$。

3.2 Square Permutation II

给定两个排列 p, q，排列长度均为奇数 n，对每个 i 决策以下三项中的**恰好一项执行**：

1. 什么都不做，花费 0 的代价。
2. 执行 $p_i \leftarrow p_i^2$ **或** $q_i \leftarrow q_i^2$，花费 x 的代价。
3. 执行 $p_i \leftarrow p_i^2$ **和** $q_i \leftarrow q_i^2$，花费 y 的代价。

m 次询问，每次给出 A, B, x, y，求最小代价使得修改后的序列 p, q 的中位数分别为 A, B，无解输出 -1。

多测，$1 \le n, m \le 10^5, 1 \le x_i \le y_i \le 10^9$。

3.2 Square Permutation II

比上一题简单多了，简单讨论下可以得出 $2x \le y$ 时 m 越小越好，否则 m 越大越好。然后做二维数点统计三类决策的数量即可。

注意要讨论 $A > n$ 且 $\sqrt{A}$ 是整数的这种情况，因为此时操作 q 的代价变成了 $y - x$。

当然你发现这些都只有 $O(1)$ 种情况，那么直接枚举就好了。

复杂度 $O(n \log n + t \log n)$。

3.3 Reconstruct the tree

给定 N 和 M 个无序点对 S，构造一棵大小为 N 的树 T 使得 T 恰好有 M 条直径，这些直径的端点构成的无序点对恰好与 S 相同。

构造方案或报告无解。

多测，$2 \le N \le 2 \times 10^5, 1 \le M \le 2 \times 10^5$。

3.3 Reconstruct the tree

首先找到所有出现在任意一条直径中的点构成的点集 S。

S 中的点一定是叶子。

3.3 Reconstruct the tree

首先找到所有出现在任意一条直径中的点构成的点集 S。

S 中的点一定是叶子。

然后我们就可以发现 S 一定能划分为若干等价类 $S = S_1 \sqcup S_2 \sqcup \ldots \sqcup S_k$，显然任意两个 $u, v \in S$，只要 u, v 不在同一个 S_i 中那么他们连出来的一定是直径。

3.3 Reconstruct the tree

首先找到所有出现在任意一条直径中的点构成的点集 S。

S 中的点一定是叶子。

然后我们就可以发现 S 一定能划分为若干等价类 $S = S_1 \sqcup S_2 \sqcup \ldots \sqcup S_k$，显然任意两个 $u, v \in S$，只要 u, v 不在同一个 S_i 中那么他们连出来的一定是直径。

证明比较容易，我们只要按照直径中心建树再判断 LCA 即可。

3.3 Reconstruct the tree

首先找到所有出现在任意一条直径中的点构成的点集 S。

S 中的点一定是叶子。

然后我们就可以发现 S 一定能划分为若干等价类 $S = S_1 \sqcup S_2 \sqcup \ldots \sqcup S_k$，显然任意两个 $u, v \in S$，只要 u, v 不在同一个 S_i 中那么他们连出来的一定是直径。

证明比较容易，我们只要按照直径中心建树再判断 LCA 即可。

构造时我们先优先压缩使用的点数，可以发现**广义菊花**是最节约的。对于 $S = S_1 \sqcup S_2 \sqcup \ldots \sqcup S_k$ 的划分，只要 $|S| + k + 1$ 个点即可构造完成，如果 $n > |S| + k + 1$，那么多的点可以放到与中心相邻的层，对答案没有影响。

3.3 Reconstruct the tree

首先找到所有出现在任意一条直径中的点构成的点集 S。

S 中的点一定是叶子。

然后我们就可以发现 S 一定能划分为若干等价类 $S = S_1 \sqcup S_2 \sqcup \ldots \sqcup S_k$，显然任意两个 $u, v \in S$，只要 u, v 不在同一个 S_i 中那么他们连出来的一定是直径。

证明比较容易，我们只要按照直径中心建树再判断 LCA 即可。

构造时我们先优先压缩使用的点数，可以发现**广义菊花**是最节约的。对于 $S = S_1 \sqcup S_2 \sqcup \ldots \sqcup S_k$ 的划分，只要 $|S| + k + 1$ 个点即可构造完成，如果 $n > |S| + k + 1$，那么多的点可以放到与中心相邻的层，对答案没有影响。

最后就是一些可能出现的特殊情况，也就是什么时候我们可以不用广义菊花。

$k = 2$ 时可以用两个中心点分隔，压掉一个点，$k = N - 1$ 时可以直接构造菊花。

Outline

1. Overview 2
2. ICPC 2025 EC Online Contest (I) 6
   2.1 Creating Chaos 7
   2.2 Robot 9
   2.3 Walk 11
3. ICPC 2025 EC Online Contest (II) 14
   3.1 Square Permutation I 15
   3.2 Square Permutation II 18
   3.3 Reconstruct the tree 20
4. ICPC 2025 Chengdu Regional Contest . 22
   4.1 Following Arrows 23
   4.2 Meeting for Meals 26
5. ICPC 2025 Nanjing Regional Contest . . 29
   5.1 Wow, It's Yesterday Six Times More 30
   5.2 What, More Kangaroos? 34
   5.3 Bucket Bonanza 36
   5.4 Trajan Algorithm 38
6. CCPC 2025 Zhengzhou Site 40
   6.1 Diameter of a Tree 41
   6.2 Subrectangle Count 43
   6.3 Parentheses and Swapping 45
7. ICPC 2025 EC Final Contest 48
   7.1 Strategic Stones 49
   7.2 Redundancy Refrain 51
   7.3 Logical Resonance 53

4.1 Following Arrows

机器人从左上角 $(1,1)$ 出发，每一步先按当前格子箭头方向移动（若会出界则不动），然后将移动前所在格子的箭头翻转为相反方向。

构造一个迷宫，使得机器人**恰好**在 k 步后首次到达右下角 (N,M)。要求 $N,M \le 8$。

多测，$1 \le k \le 10^6$。

4.1 Following Arrows

考虑先构造一个能让答案达到 10^6 的迷宫。

大概长这样（官方 std 中的某个例子）：

```
RRRRRRRD
DDLLLLLL
DRRRRRRD
DDLLLLLL
DRRRRRRD
DDLLLLLL
DRRRRRRD
DDLLLLLL
```

4.1 Following Arrows

我们如果翻转所有格子就能得到一个 10^6 量级的答案。

4.1 Following Arrows

我们如果翻转所有格子就能得到一个 10^6 量级的答案。

此外我们发现由于棋盘最终状态的格子一定与初始状态相反，那么也就是说我们在初始棋盘上进行的每个翻转都独立（当然是对于路径上的）。

4.1 Following Arrows

我们如果翻转所有格子就能得到一个 10^6 量级的答案。

此外我们发现由于棋盘最终状态的格子一定与初始状态相反，那么也就是说我们在初始棋盘上进行的每个翻转都独立（当然是对于路径上的）。

这样我们背包把所有的 k 都背出来就行了，当然有些值可能取不到，需要多换几个初始状态。

另外 k 比较小的时候也取不到，需要进行一些特判。

4.2 Meeting for Meals

给定一张 n 个点 m 条边的带权无向连通图，图上有 k 个人，初始在 $a_1, a_2, \ldots, a_k$，所有人速度均为 1（人可以选择站在原地不动）。这 k 个人要去 1 号点会合，所有人都抵达 1 号点时的时间记为 T，记所有方案中最小的 T 为 T_m。

你要在维持 $T = T_m$ 的情况下，**独立的**回答每个人最长能有多少时间与别人结伴而行（这里的结伴而行是指他们的位置**完全重合**）。

多测，$2 \le k \le n \le 3 \times 10^5, n - 1 \le m \le 10^6$，$a_i$ 互不相同。

4.2 Meeting for Meals

先跑个从 1 出发的单源最短路求 T_m，然后我们就可以先不管这个东西了。

因为显然对于现在正在求答案的 i，当 i 和 j 相遇之后就不会再分开了，所有最大化有人结伴而行的时间正是最小化首次与人相遇的时间。

4.2 Meeting for Meals

先跑个从 1 出发的单源最短路求 T_m，然后我们就可以先不管这个东西了。

因为显然对于现在正在求答案的 i，当 i 和 j 相遇之后就不会再分开了，所有最大化有人结伴而行的时间正是最小化首次与人相遇的时间。

我们现在的问题就变为了：i 和谁相遇能有贡献？i 和别人在哪相遇能有贡献？

这两个问题看似很难，实则一点也不简单。

4.2 Meeting for Meals

先跑个从 1 出发的单源最短路求 T_m，然后我们就可以先不管这个东西了。

因为显然对于现在正在求答案的 i，当 i 和 j 相遇之后就不会再分开了，所有最大化有人结伴而行的时间正是最小化首次与人相遇的时间。

我们现在的问题就变为了：i 和谁相遇能有贡献？i 和别人在哪相遇能有贡献？

这两个问题看似很难，实则一点也不简单。

考虑到我们均摊下来每个节点应该只能保留 $O(1)$ 的信息，我们猜测每个节点保留两个最近的人即可计算出所有的答案。

4.2 Meeting for Meals

首先每个人初始位置互不相同，所以一定会贡献至少一次。对于节点 u，若 j 比 i 先到达节点 u，那么 i 一定不会选择经过 u 再去与其他人碰面，因为此时 i 不如直接选择让 j 在 u 等他（虽然最优答案不会这么去算）。

所以每个节点只有**距离它最近的人**向后转移才有意义，那么我们这里保留了最近和次进，就正好可以计算答案了。

当然两个人可能在某条边上相遇，这种情况的转移也要讨论下，此外可以直接看到答案一定是 0.5 的整数倍，随便乘个 2 拿整数做完再除回来即可。

复杂度 $O((n+m)\log n)$。

Outline

1. Overview 2
2. ICPC 2025 EC Online Contest (I) 6
   2.1 Creating Chaos 7
   2.2 Robot 9
   2.3 Walk 11
3. ICPC 2025 EC Online Contest (II) 14
   3.1 Square Permutation I 15
   3.2 Square Permutation II 18
   3.3 Reconstruct the tree 20
4. ICPC 2025 Chengdu Regional Contest . 22
   4.1 Following Arrows 23
   4.2 Meeting for Meals 26
5. ICPC 2025 Nanjing Regional Contest . . 29
   5.1 Wow, It's Yesterday Six Times More 30
   5.2 What, More Kangaroos? 34
   5.3 Bucket Bonanza 36
   5.4 Trajan Algorithm 38
6. CCPC 2025 Zhengzhou Site 40
   6.1 Diameter of a Tree 41
   6.2 Subrectangle Count 43
   6.3 Parentheses and Swapping 45
7. ICPC 2025 EC Final Contest 48
   7.1 Strategic Stones 49
   7.2 Redundancy Refrain 51
   7.3 Logical Resonance 53

5.1 Wow, It's Yesterday Six Times More

交互题，给定 $n \times m$ 的袋鼠棋盘，棋盘上某个位置 (x, y) 有一个洞，其他位置初始都恰好有一个袋鼠。

你可以调用 UDLR 四种操作操控袋鼠，如果袋鼠移动到棋盘外或洞上那么它们就会掉出去，交互器会返回这一轮掉出去的袋鼠数量。

你需要在至多 $n + m + 10$ 次操作内确定洞的位置。

多测，$3 \le n, m \le 30$。

5.1 Wow, It's Yesterday Six Times More

先考虑持续往上推，确定洞的纵坐标。

5.1 Wow, It's Yesterday Six Times More

先考虑持续往上推，确定洞的纵坐标。

我们发现一直执行 U，掉下去的袋鼠只有 $m+1$ 或 m 两种（如果 n 为奇数且洞在中间可能会出现 $m-1$），当掉下去 m 个袋鼠，那么说明有两种情况：

```
1  KK.KK        KKKKK
2  KKxKK        KK.KK
3  KKKKK        KKxKK
4  .....        .....
```

它们无法区分。

如果洞在 n 偶数中间的两行或者 n 奇数中间的一行，那么这种情况比较简单，单独讨论即可。

5.1 Wow, It's Yesterday Six Times More

接下来讨论其他情况，我们先通过等量的 D 把袋鼠移动回来：

```
1  ....        ....
2  .H..        ....
3  K.KK        KKKK
4  K.KK        KHKK
5  KKKK        K.KK
```

若洞不在第一行或最后一行，那么我们后续只要一直执行 L，当掉下去的袋鼠数**第一次**小于 $n-x$（x 为执行 U 的次数）时，那么就固定了列的位置，剩下洞在上半或下半的情况也容易讨论。

5.1 Wow, It's Yesterday Six Times More

最后就是洞在第一行或最后一行的情况了：

```
.H..        ....
K.KK        KKKK
KKKK        KHKK
```

这种情况也是执行 L，若第一步是 n 则说明洞在下面，U 一次一直 L 即可。否则 L $m-1$ 次，若出现 $n-2$ 则确定位置，否则只有三种情况，RDL 就做完了：

```
..H     ...     ...
...     K..     K..
K..     H..     ..H
```

5.2 What, More Kangaroos?

一条数轴上有 n 只袋鼠，其中第 i 只袋鼠最初位于**正坐标** c_i，并且有两种步长 a_i 和 b_i。

你有四个按钮，你可以按下它们任意多次，每个按钮的功能如下：

1. $\forall i, c_i \leftarrow c_i + a_i$。
2. $\forall i, c_i \leftarrow c_i - a_i$。
3. $\forall i, c_i \leftarrow c_i + b_i$。
4. $\forall i, c_i \leftarrow c_i - b_i$。

问你最多能让多少袋鼠移动到**负坐标**（$c_i < 0$）。

多测，$1 \le n \le 2 \times 10^5, -10^9 \le a_i, b_i \le 10^9, 1 \le c_i \le 10^9$。

5.2 What, More Kangaroos?

形式化题意为找到整点 (x, y) 使得 $a_i x + b_i y + c_i < 0$ 的 i 最多。

这些区域都是半平面，做不了一点。

5.2 What, More Kangaroos?

形式化题意为找到整点 (x, y) 使得 $a_i x + b_i y + c_i < 0$ 的 i 最多。

这些区域都是半平面，做不了一点。

但是我们发现这个问题的答案和令 $c_i = 0$ 的问题的答案一致，因为总存在一个 $M > 0$ 使得若 $a_i x + b_i y < 0$ 则 $M a_i x + M b_i y + c_i < 0$（$n$ 有限）。

这样就是比较简单的过原点的半平面了，直接极角排序再双指针即可即可。

注意这里用到了**整点极角的稠密性**，不过也比较简单。

5.3 Bucket Bonanza

给你 n 个桶，每个桶有容量 v_i 和每秒漏水量 l_i 两个参数，你需要进行 q 次独立的游戏，每次游戏你需要先进行任意次的水桶合并：v 取 max，l 取 min，将水桶装满水漏 t_i 秒，最大化 t_i 秒后水桶剩余水量的总和。

多测，$1 \le n, q \le 2 \times 10^5, 1 \le v_i \le 4 \times 10^{13}, 0 \le l_i \le 10^9, 0 \le t_i \le 10^9$。

5.3 Bucket Bonanza

考虑答案为 $\sum v - t_i \sum l$，那么可以拆求和符号，让小的 v 匹配大的 l，例如在 $\min v - t_i \max l$ 时，应该想办法通过合并让这一项消失。

5.3 Bucket Bonanza

考虑答案为 $\sum v - t_i \sum l$，那么可以拆求和符号，让小的 v 匹配大的 l，例如在 $\min v - t_i \max l$ 时，应该想办法通过合并让这一项消失。

我们发现这非常简单，若 $\min v$ 和 $\max l$ 在同一个桶，那么随便把它和另一个合并，否则不是同一个桶，直接把它们合并即可。

最后答案可以离线下来做，也可以直接二分这个消去的长度，复杂度 $O(q \log n)$。

5.4 Trajan Algorithm

在点双的 Tarjan 算法中，写错了几行，你需要判断对于每个节点 u，问是否存在一个节点 v 和一个二维数组 adj（adj 必须表示给定的图），使得 $\text{tarjan}(v, 0)$ 和 $\text{trajan}(v, 0)$ 在节点 u 上产生不同的结果。

Algorithm 1 The Correct Tarjan Algorithm

```
function TARJAN(i, d)
    visited[i] ← true
    depth[i] ← d
    low[i] ← d
    childCount ← 0
    isArticulation ← false

    for each j in adj[i] do
        if j = parent[i] then
            continue
        end if
        if not visited[j] then
            parent[j] ← i
            TARJAN(j, d + 1)
            childCount ← childCount + 1
            if low[j] ≥ depth[i] then
                isArticulation ← true
            end if
            low[i] ← min(low[i], low[j])
        else
            low[i] ← min(low[i], depth[j])
        end if
    end for
    if (parent[i] ≠ null and isArticulation)
  or (parent[i] = null and childCount > 1)
  then
        Output i as articulation point
    end if
end function
```

Algorithm 2 The Incorrect Trajan Algorithm

```
function TRAJAN(i, d)
    visited[i] ← true
    depth[i] ← d
    low[i] ← d
    childCount ← 0
    isArticulation ← false

    for each j in adj[i] do
        if j = parent[i] then
            continue
        end if
        if not visited[j] then
            parent[j] ← i
            TRAJAN(j, d + 1)
            childCount ← childCount + 1
            if low[j] ≥ depth[i] then
                isArticulation ← true
            end if
        end if
        low[i] ← min(low[i], low[j])


    end for
    if (parent[i] ≠ null and isArticulation)
  or (parent[i] = null and childCount > 1)
  then
        Output i as articulation point
    end if
end function
```

多测，$2 \le n \le 10^5, 1 \le m \le 10^5$，保证图连通，没有重边自环。

5.4 Trajan Algorithm

考虑 Trajan 算法只会让 low 变小，所以**绝对不可能**把不是割点的判成割点。

那么割点是怎么被判错的呢？

5.4 Trajan Algorithm

考虑 Trajan 算法只会让 low 变小，所以**绝对不可能**把不是割点的判成割点。

那么割点是怎么被判错的呢？

时刻记住无向图没有 **Cross** 边。

显然若 u 为根，那么割点一定不会改变，所以 u 一定不是根。

5.4 Trajan Algorithm

考虑 Trajan 算法只会让 low 变小，所以**绝对不可能**把不是割点的判成割点。

那么割点是怎么被判错的呢？

时刻记住无向图没有 **Cross** 边。

显然若 u 为根，那么割点一定不会改变，所以 u 一定不是根。

那么就很容易了，首先 u 从 fa 过来，然后先有一条回到 fa 所在点双的边去更新 low，然后它所链接的所有其他点双都均有一条 **Tree** 边和至少一条 **Back** 边相连。

那么判定条件也就是**连接至少两个点双且不是任意一条割边的端点**。

复杂度 $O(n+m)$。

Outline

1. Overview 2
2. ICPC 2025 EC Online Contest (I) 6
   2.1 Creating Chaos 7
   2.2 Robot 9
   2.3 Walk 11
3. ICPC 2025 EC Online Contest (II) 14
   3.1 Square Permutation I 15
   3.2 Square Permutation II 18
   3.3 Reconstruct the tree 20
4. ICPC 2025 Chengdu Regional Contest . 22
   4.1 Following Arrows 23
   4.2 Meeting for Meals 26
5. ICPC 2025 Nanjing Regional Contest . . 29
   5.1 Wow, It's Yesterday Six Times More 30
   5.2 What, More Kangaroos? 34
   5.3 Bucket Bonanza 36
   5.4 Trajan Algorithm 38
6. CCPC 2025 Zhengzhou Site 40
   6.1 Diameter of a Tree 41
   6.2 Subrectangle Count 43
   6.3 Parentheses and Swapping 45
7. ICPC 2025 EC Final Contest 48
   7.1 Strategic Stones 49
   7.2 Redundancy Refrain 51
   7.3 Logical Resonance 53

6.1 Diameter of a Tree

给定一棵大小为 n 的树 T，重排点的编号使得**字典序最大的直径字典序最小**。

多测，$2 \le n \le 2 \times 10^5$。

6.1 Diameter of a Tree

记可能是直径端点的叶子有 k 个，然后令直径中点为根。

现在端点肯定用 $1 \sim k$，并且字典序最大的肯定是 k 起步。

那么前半段一定是 $k, k+1, k+2, \ldots$。

6.1 Diameter of a Tree

记可能是直径端点的叶子有 k 个，然后令直径中点为根。

现在端点肯定用 $1 \sim k$，并且字典序最大的肯定是 k 起步。

那么前半段一定是 $k, k+1, k+2, \ldots$。

后半段我们发现是要尽可能减少兄弟的数量让自身尽量小，也就是要选择 deg 字典序最小的路径，这个路径恰好可以通过 bfs 实现，只需每次仅保留当前最优解即可。

特别注意最后一个端点的范围与其他的不同，直径为偶数的处理方式差不多，复杂度 $O(n)$。

6.2 Subrectangle Count

给定一个长度为 n 的序列 a_i 和一个长度为 m 的序列 b_i，构造 $c_{i,j} = a_i \oplus b_j$，其中 $\oplus$ 是按位异或。求有多少有序数对 (i, j) 满足 $c_{i,j} + 3 = c_{i,j+1} + 2 = c_{i+1,j} + 1 = c_{i+1,j+1}$。

多测，$2 \le n, m \le 2 \times 10^5, 0 \le a_i, b_j < 2^{30}$。

6.2 Subrectangle Count

记 $A = a_i \oplus a_{i+1}, B = b_j \oplus b_{j+1}, X = c_{i,j}$。

那么也就是要求 $B = X \oplus (X+1) = (X+2) \oplus (X+3), A = X \oplus (X+2) = (X+1) \oplus (X+3)$。

6.2 Subrectangle Count

记 $A = a_i \oplus a_{i+1}, B = b_j \oplus b_{j+1}, X = c_{i,j}$。

那么也就是要求 $B = X \oplus (X+1) = (X+2) \oplus (X+3), A = X \oplus (X+2) = (X+1) \oplus (X+3)$。

显然 $A = 11...10, B = 11...1$，那么明显若 $B \neq 1$ 且 $B = X \oplus (X+1)$，那么 $B \neq (X+2) \oplus (X+3)$，所以 $B = 1$。

6.2 Subrectangle Count

记 $A = a_i \oplus a_{i+1}, B = b_j \oplus b_{j+1}, X = c_{i,j}$。

那么也就是要求 $B = X \oplus (X+1) = (X+2) \oplus (X+3), A = X \oplus (X+2) = (X+1) \oplus (X+3)$。

显然 $A = 11...10, B = 11...1$，那么明显若 $B \neq 1$ 且 $B = X \oplus (X+1)$，那么 $B \neq (X+2) \oplus (X+3)$，所以 $B = 1$。

那么只有 A 的部分就是把 $011...10 \oplus 111...10 = 100...00$ 完成 $+2$，拿 Trie 树随便做一下就行。

复杂度 $O((n+m)\log V)$。

6.3 Parentheses and Swapping

给定一个长度为 n 的序列 a，要去构造一个合法括号序列 b，记 p_i 表示在 b 序列中与第 i 个括号匹配的另一半括号的下标，使得序列 $c_i = a_{p_i}$ 的字典序最小。

多测，$2 \le n \le 5 \times 10^5, 1 \le a_i \le n$，$n$ 是偶数。

6.3 Parentheses and Swapping

考虑贪心，如果所有 a 互不相同那么决策就很少，具体来说枚举 i 从 1 到 n，维护左括号的栈：

- 若 i 没有被钦定为右括号，那么 $b_i = ($，且匹配的 p_i 应是与 i 奇偶性不同的、在栈中上一个匹配到的右括号之前的、a_j 最小的 j。
- 否则 $b_i =)$，弹栈。

6.3 Parentheses and Swapping

考虑贪心，如果所有 a 互不相同那么决策就很少，具体来说枚举 i 从 1 到 n，维护左括号的栈：

- 若 i 没有被钦定为右括号，那么 $b_i = ($，且匹配的 p_i 应是与 i 奇偶性不同的、在栈中上一个匹配到的右括号之前的、a_j 最小的 j。
- 否则 $b_i =)$，弹栈。

那我们发现 a 有相同的好像也差不多，为了让后续左括号能匹配的位置尽可能多，我们尽量选择靠后的 j。

6.3 Parentheses and Swapping

然后现在一个位置 i 就有可能能选择是左括号还是右括号，考虑如果能选择，要求如下（记栈顶为 s）：

- i 不是 p_s（这是最靠后的了，必须是右括号）。
- $a_i = a_{p_s}$。
- $a_s = a_j$，其中 j 是后续可以匹配的与 i 奇偶性不同的、在栈中上一个匹配到的右括号之前的、a_j 最小的 j。

这样的话我们根据贪心策略，要让后续匹配的限制尽量少，应该选择 $b_i =)$，因为如果我们让他往后匹配会得到 (A(B)C)，而往前匹配得到的 (A)B(C) 仍是合法括号串，但后者对 $B(C)$ 的匹配方式限制更少！

这样贪心即可解决，复杂度 $O(n \log n)$。

Outline

1. Overview 2
2. ICPC 2025 EC Online Contest (I) 6
   2.1 Creating Chaos 7
   2.2 Robot 9
   2.3 Walk 11
3. ICPC 2025 EC Online Contest (II) 14
   3.1 Square Permutation I 15
   3.2 Square Permutation II 18
   3.3 Reconstruct the tree 20
4. ICPC 2025 Chengdu Regional Contest . 22
   4.1 Following Arrows 23
   4.2 Meeting for Meals 26
5. ICPC 2025 Nanjing Regional Contest . . 29
   5.1 Wow, It's Yesterday Six Times More 30
   5.2 What, More Kangaroos? 34
   5.3 Bucket Bonanza 36
   5.4 Trajan Algorithm 38
6. CCPC 2025 Zhengzhou Site 40
   6.1 Diameter of a Tree 41
   6.2 Subrectangle Count 43
   6.3 Parentheses and Swapping 45
7. ICPC 2025 EC Final Contest 48
   7.1 Strategic Stones 49
   7.2 Redundancy Refrain 51
   7.3 Logical Resonance 53

7.1 Strategic Stones

给定一个长度为 n 的 01? 串，A 和 B 在玩一个博弈游戏，A 先手。

A 可以把某个 ? 改成 0，B 可以把某个 ? 改成 1，A 要最大化最长连续的 0 的数量，B 要最小化最长连续的 0 的数量。

问在两人都足够聪明的情况下最终最长的连续的 0 的数量是多少。

多测，$1 \le n \le 10^5$。

7.1 Strategic Stones

首先先简化下题目，1 会把字符串分为很多独立的段，每段中又会被？分为很多段，记 l_i 表示第 $i-1$ 个问号和第 i 个问号之间的 0 的数量。

7.1 Strategic Stones

首先先简化下题目，1 会把字符串分为很多独立的段，每段中又会被？分为很多段，记 l_i 表示第 $i-1$ 个问号和第 i 个问号之间的 0 的数量。

显然每次 A 的操作是 $l \leftarrow l_{i-1} + l_i + 1$，而 B 一定会贴着 A 这次操作得到的 0 连续段放 1。

考虑 $l = [6, 2, 2, 2, 2, 6]$ 的答案。

7.1 Strategic Stones

首先先简化下题目，1 会把字符串分为很多独立的段，每段中又会被？分为很多段，记 l_i 表示第 $i-1$ 个问号和第 i 个问号之间的 0 的数量。

显然每次 A 的操作是 $l \leftarrow l_{i-1} + l_i + 1$，而 B 一定会贴着 A 这次操作得到的 0 连续段放 1。

考虑 $l = [6, 2, 2, 2, 2, 6]$ 的答案。

是 11，因为 A 合并左侧和右侧的两个 2 时 B 为了不让答案比 11 更大只能被迫堵 6 那侧，然后 A 再合并两个 5。

同理 $l = [6, 2, 2, 2, 2, 2, 2, 6]$ 也是 11。

7.1 Strategic Stones

首先先简化下题目，1 会把字符串分为很多独立的段，每段中又会被？分为很多段，记 l_i 表示第 $i-1$ 个问号和第 i 个问号之间的 0 的数量。

显然每次 A 的操作是 $l \leftarrow l_{i-1} + l_i + 1$，而 B 一定会贴着 A 这次操作得到的 0 连续段放 1。

考虑 $l = [6, 2, 2, 2, 2, 6]$ 的答案。

是 11，因为 A 合并左侧和右侧的两个 2 时 B 为了不让答案比 11 更大只能被迫堵 6 那侧，然后 A 再合并两个 5。

同理 $l = [6, 2, 2, 2, 2, 2, 2, 6]$ 也是 11。

那么现在就很简单了，先二分，然后这样我们就能通过递推得到当 A 操作这个问号时 B 是不是一定要堵住左侧，最终就可以完成最难的这部分转移。

复杂度 $O(n \log n)$。

7.2 Redundancy Refrain

给定一个长度为 n 的序列，你要把这里面出现的 0 全都改为 $1 \sim k$ 之间的整数。使得最长的**构成多重集相同的**两段长度相同的区间长度最短。

构造方案。

多测，$2 \le n \le 2 \times 10^5, 1 \le k \le n$。

7.2 Redundancy Refrain

显然两段区间相交，否则若 ABC 是答案，那么 ABC 是更大的答案。

7.2 Redundancy Refrain

显然两段区间相交，否则若 ABC 是答案，那么 ABC 是更大的答案。

那么同理，两段相交的区间的答案能被我们映射回两段不交的区间的答案，并且答案只与这两段区间的左端点之差相关。

那么我们怎么让**左端点之差**尽量大呢？

7.2 Redundancy Refrain

显然两段区间相交，否则若 ABC 是答案，那么 ABC 是更大的答案。

那么同理，两段相交的区间的答案能被我们映射回两段不交的区间的答案，并且答案只与这两段区间的左端点之差相关。

那么我们怎么让**左端点之差**尽量大呢？

答案就是原区间**长度为 1**！

那么我们构造的目的就是最小化两个相同的数之间的最大距离。这个我相信大家手拿把掐，二分完随便贪一下即可。

复杂度 $O(n \log n)$。

7.3 Logical Resonance

通信题，你需要建立一个 A 类型有根树到 B 类型有根树的双向映射，其中 A,B 类型定义如下：

- A 类型满足 1 是根且 $p_i < i$。
- B 类型满足 n 是根且 $q_i > i$。

此外，你还需要保证你建立的映射 $T_A \Longleftrightarrow T_B$ 满足 T_A 中的所有叶子在 T_B 中均不是叶子，T_A 中的所有非叶子在 T_B 中均是叶子。

多测，$2 \le n \le 10^3$。

7.3 Logical Resonance

其实有点像你在寻找一个树对偶的过程。

7.3 Logical Resonance

其实有点像你在寻找一个树对偶的过程。

考虑一个非常精妙的递归做法。先考虑 $T_A \Longrightarrow T_B$，$T_B \Longrightarrow T_A$ 可以直接对称。

7.3 Logical Resonance

其实有点像你在寻找一个树对偶的过程。

考虑一个非常精妙的递归做法。先考虑 $T_A \Longrightarrow T_B$，$T_B \Longrightarrow T_A$ 可以直接对称。

每次找到**编号最小的叶子** u，将其父亲在 B 类型中的父亲设为它（就是 $q_{p_u} = u$）。然后把 p_u 用 u 替换得到一棵大小为 $n-1$ 的树，然后重复执行直到只剩一个点。

精细一点实现可以做到 $O(n\alpha(n))$，但是没必要。

7.3 Logical Resonance

其实有点像你在寻找一个树对偶的过程。

考虑一个非常精妙的递归做法。先考虑 $T_A \Longrightarrow T_B$，$T_B \Longrightarrow T_A$ 可以直接对称。

每次找到**编号最小的叶子** u，将其父亲在 B 类型中的父亲设为它（就是 $q_{p_u} = u$）。然后把 p_u 用 u 替换得到一棵大小为 $n-1$ 的树，然后重复执行直到只剩一个点。

精细一点实现可以做到 $O(n\alpha(n))$，但是没必要。

至于证明，我们需要先换一个缩边的顺序：每次找到**编号最小且缩边后仍满足 $p_i < i$ 的叶子** u 进行缩边。

首先这不会改变得到的树的最终形态，然后只需要说明这种情况下得到的两棵树不可能相同即可，严格证明有些长了，这里写不下。

当然官方题解里给出了两种不同的做法，感兴趣也可以看一看。

Thanks!